

MESH: MEMORY-EFFICIENT SINKHORN OPTIMIZATION FOR MIXTURE-OF-EXPERTS TRAINING

Masato Fujitake

FA Research, Fast Accounting Co., Ltd.

fujitake@fastaccounting.co.jp

ABSTRACT

Memory-efficient matrix optimizers such as Sinkhorn gradient descent remove most AdamW optimizer state for dense Transformer matrices, but direct application to Mixture-of-Experts (MoE) training is unreliable. We study this failure in a controlled 110M-parameter nanowhale DeepSeek-style MoE pretraining setting. A SAGE/Sinkhorn hybrid reduces optimizer state from 0.883GB to 0.331GB but degrades evaluation loss to 3.8265, far above the AdamW baselines observed in the same setup (3.58–3.64 across the seeds we study). We show that routed MoE expert matrices are the dominant failure point: their gradients are conditional, temporally varying, and poorly served by stateless Sinkhorn normalization. We propose MESH, a hidden-momentum Sinkhorn update for MoE experts. MESH restores a temporal first-moment signal through the gradient-buffer lifecycle, without storing the expert first moment as optimizer state. MESH-B is an optional block-preconditioned variant that adds a coarse neuron/block inverse-RMS multiplier. Across ablations, temporal smoothing before matrix normalization is the primary causal ingredient; block/neuron preconditioning can improve the memory-quality frontier, but is not established as universally necessary. In two additional seeds, MESH and MESH-B reduce optimizer-state memory by 62.5% and peak PyTorch CUDA allocation by about 12.6% relative to AdamW, with a modest evaluation-loss gap. Full-state diagnostic variants recover AdamW-like performance in ablations, supporting the conclusion that MoE experts need temporal smoothing, but not necessarily full coordinate-wise AdamW state.

Index Terms— Mixture-of-Experts, memory-efficient optimization, Sinkhorn optimization, hidden momentum, language model pretraining

1. INTRODUCTION

AdamW remains the default optimizer for language-model pretraining because it combines temporal first-moment smoothing with coordinate-wise adaptive scaling [1, 2]. Its memory cost, however, is substantial: AdamW stores two full state tensors per trainable parameter. This motivates optimizers that reduce or remove moment state. Sinkhorn-style matrix

optimizers normalize gradient matrices by alternating row and column operations, yielding stateless updates for dense two-dimensional weights [4]. SAGE-style hybrids further recognize that embeddings are difficult for purely stateless methods and assign light-state updates to vocabulary tensors while leaving dense matrices to Sinkhorn [5].

These role assignments were developed mainly for dense Transformer settings. We ask whether they transfer to Mixture-of-Experts (MoE) language models. The answer is negative: a routed expert matrix is two-dimensional, but its gradient is not the same object as a dense MLP gradient. Each expert receives a router-selected token subset, weighted by learned gate scores, and this conditional distribution changes across training. In a nanowhale 110M DeepSeek-style MoE model, a direct SAGE/Sinkhorn hybrid lowers optimizer state to 0.331GB but reaches only 3.8265 evaluation loss, far worse than AdamW. Applying SAGE to experts is worse still, and reducing Sinkhorn scale, fusing expert tensors, or using route-confidence damping does not close the gap.

The key question is therefore not whether Sinkhorn can be memory-efficient, but what it lacks for routed MoE experts. Our ablations indicate that the primary missing ingredient is temporal smoothing before matrix normalization. Stateless Sinkhorn computes $\text{Sinkhorn}(G_t)$ from the instantaneous expert gradient. In a routed expert, G_t is a high-variance, conditional sample of an expert-specific token distribution. Because Sinkhorn is nonlinear, $\text{Sinkhorn}(\mathbb{E}[G_t])$ and $\mathbb{E}[\text{Sinkhorn}(G_t)]$ can behave differently; the latter amplifies step-to-step routing noise. The optimizer failure is an ordering problem: normalize-then-average behaves differently from average-then-normalize. MESH converts the update to $\text{Sinkhorn}(H_t)$, where H_t averages the expert’s gradient history before matrix normalization.

We propose MESH, a MoE-aware, hidden-momentum Sinkhorn optimizer. MESH keeps the SAGE/Sinkhorn hybrid for vocabulary and non-expert matrices, but replaces stateless expert Sinkhorn with hidden-momentum expert Sinkhorn. Its expert first moment is maintained through the gradient-buffer lifecycle rather than as an optimizer-state tensor. We also evaluate MESH-B, which adds a coarse block/neuron inverse-RMS multiplier. The main empirical message is deliberately conservative: MoE experts primarily

need temporal smoothing before matrix normalization; block-level adaptivity can help, especially in seed-42 diagnostics, but is not the sole causal ingredient.

Our contributions are:

- We identify routed MoE experts as the failure point of direct SAGE/Sinkhorn optimization in nanowhale pre-training.
- We introduce MESH, a hidden-momentum Sinkhorn expert update that restores a first-moment signal without adding explicit expert first-moment optimizer state.
- We evaluate MESH-B, an optional block-preconditioned extension, and show that the unblocked and block-preconditioned hidden-momentum variants give similar multi-seed memory-quality trade-offs.
- We provide negative ablations showing that second-moment-only, sign/momentum-only, confidence-only, scale-only, and fused-Sinkhorn interventions do not explain the gap.

2. RELATED WORK

2.1. Adaptive and memory-efficient optimization

Adam and AdamW maintain first- and second-moment estimates for every parameter [1, 2]. This robustness costs two state tensors per parameter. Memory-efficient Adam variants reduce this cost in different ways: Adam-mini uses fewer learning-rate resources by exploiting block structure [7], and HMAAdam/HMAAdamW remove the explicit first-moment tensor by storing a momentum-like quantity in the gradient buffer [6]. MESH uses the latter idea only for MoE expert first moments and combines it with Sinkhorn-style matrix normalization.

2.2. Matrix normalization and SAGE hybrids

Sinkhorn and gradient multi-normalization optimizers compute matrix updates through row and column normalization [3, 4]. Their appeal is that a large dense matrix can be updated without storing AdamW’s two full state tensors. SAGE extends this strategy by treating embeddings as a separate difficult class [5]. Our work identifies a second difficult class: routed MoE expert matrices.

2.3. Mixture-of-Experts models

Sparse MoE layers route tokens to a subset of experts, increasing model capacity without activating every parameter on every token [8–10]. This routing changes the optimizer problem. Expert gradients are conditional on router decisions and gate weights, so a routed expert matrix is not equivalent to a dense MLP matrix even when it has the same shape. MESH is designed for this expert-specific gradient structure.

3. BACKGROUND AND DIAGNOSIS

3.1. Sinkhorn updates for dense matrices

Let $W \in \mathbb{R}^{m \times n}$ be a matrix parameter and $G_t = \nabla_W \mathcal{L}_t$ its gradient at step t . A Sinkhorn-style matrix optimizer forms an update direction by repeatedly normalizing row and column norms,

$$U_t = \text{Sinkhorn}(G_t; K, \gamma), \quad (1)$$

where K is the number of normalization rounds and γ is a scalar update scale. This update is attractive because it is stateless: it uses the current matrix gradient and stores no first or second moment for dense matrices.

3.2. Why routed experts are not ordinary dense matrices

A routed MoE expert weight is also a matrix, but its gradient has a different sampling structure. In a top- k MoE layer, expert e receives only tokens selected by the router:

$$G_{e,t} = \sum_{i: e \in \text{topk}(x_i, t)} a_{i,e,t} g_{i,e,t}, \quad (2)$$

where $a_{i,e,t}$ is the gate weight. Thus $G_{e,t}$ is a gradient under the conditional distribution $p(x \mid e \in \text{topk}(x, t))$, not under the full mini-batch distribution. This conditional distribution changes as the router and hidden states change during training.

This matters because Sinkhorn is nonlinear. Dense training can often tolerate $\text{Sinkhorn}(G_t)$ because G_t is a reasonably stable estimate of a full-batch dense-matrix gradient. In routed experts, however, the sequence $G_{e,t}$ carries routing-induced temporal noise. Applying Sinkhorn before smoothing computes $\text{Sinkhorn}(G_{e,t})$ for each noisy expert sample. Momentum instead approximates applying Sinkhorn to a temporally averaged expert signal. This distinction is the central diagnosis of the paper.

3.3. Hypotheses we test

We considered several explanations for the expert failure. Raw route-count sparsity could reduce effective batch size; gate-weight skew could produce effective sparsity even when every expert is selected; semantic routing could make expert gradients non-i.i.d.; Sinkhorn could be sensitive to expert tensor partitioning; or stateless Sinkhorn could simply lack temporal smoothing. Our experiments rule out the first four as complete explanations. The remaining supported hypothesis is that routed experts require a first-moment signal before matrix normalization. Coarse block/neuron preconditioning can improve this update, but the multi-seed results indicate that the first moment is the primary causal ingredient.

4. MESH

4.1. Problem: MoE expert gradients are not dense-matrix gradients

Let W_e be an expert matrix and $G_{e,t} = \nabla_{W_e} \mathcal{L}_t$ its gradient at step t . A dense MLP matrix receives gradient contributions from essentially all tokens in a minibatch. A routed expert instead receives

$$G_{e,t} = \sum_{i \in \mathcal{R}_{e,t}} a_{i,e,t} g_i(W_e), \quad (3)$$

where $\mathcal{R}_{e,t}$ is the token subset selected for expert e and $a_{i,e,t}$ is the gate weight. Both the subset and the weights change with the router and hidden states. Thus the expert gradient is a temporally varying conditional estimate. Applying stateless Sinkhorn directly to $G_{e,t}$ uses

$$U_{e,t} = \text{Sinkhorn}(G_{e,t}; K, \tau), \quad (4)$$

which normalizes the instantaneous matrix without remembering whether the expert signal is persistent or a one-step routing fluctuation.

4.2. MESH: hidden momentum before Sinkhorn

MESH changes only the MoE expert role. The primary algorithm, MESH, maintains an unnormalized expert momentum accumulator

$$H_{e,t} = \beta_1 H_{e,t-1} + G_{e,t}. \quad (5)$$

This differs by a constant scale from the conventional Adam first moment. We use $H_{e,t}$ directly because Sinkhorn normalization and mean-normalized block multipliers are invariant to global scaling up to the explicit update scale. The expert matrix direction is then

$$U_{e,t} = \text{Sinkhorn}(H_{e,t}; K, \tau), \quad (6)$$

where K is the number of alternating row/column normalization rounds and τ is the Sinkhorn update scale.

The hidden-momentum implementation stores $H_{e,t}$ outside the optimizer-state dictionary. After an optimizer step, the expert gradient buffer is stashed. Immediately before the next backward pass, the buffer is reattached as $p.grad \leftarrow \beta_1 H_{e,t}$. Backward accumulation adds the current gradient, yielding $H_{e,t+1}$. This prepare-before-backward lifecycle was necessary in Trainer/Accelerate/DDP-style loops, where a naive `zero_grad` bridge can be bypassed by wrapped-model gradient clearing. Diagnostics in our final implementation report approximately one prepare call per optimizer step, buffer-shadow cosine near one, and relative error around 10^{-8} .

4.3. MESH-B: optional block/neuron scaling

MESH can be used as hidden-momentum Sinkhorn alone. We also evaluate a block-scaled variant, denoted MESH-B, which applies a coarse inverse-RMS multiplier to the Sinkhorn direction. For expert w_1, w_3 matrices we use row statistics; for w_2 we use column statistics. Let $R(H_{e,t})$ denote this block RMS. The default MESH-B statistic is computed from the smoothed source $H_{e,t}$ rather than a separate raw-gradient stream:

$$V_{e,t} = \beta_2 V_{e,t-1} + (1 - \beta_2) R(H_{e,t})^2. \quad (7)$$

We use H for this statistic because the best seed-42 diagnostic used $V(H)$, and because it keeps the hidden-momentum implementation simple without storing an additional raw-gradient stream. Raw-gradient block statistics are a natural alternative, but we leave a systematic comparison to future work.

The multiplier is clipped and mean-normalized,

$$Q_{e,t} = \text{clip} \left(\frac{(V_{e,t} + \epsilon)^{-p/2}}{\text{mean}((V_{e,t} + \epsilon)^{-p/2})}, q_{\min}, q_{\max} \right), \quad (8)$$

and the expert update is

$$W_{e,t+1} = W_{e,t} - \eta Q_{e,t} \odot U_{e,t} - \eta \lambda W_{e,t}, \quad (9)$$

with $Q_{e,t}$ broadcast over the appropriate block dimension. The block multiplier is intentionally coarse. It is not a full coordinate-wise second moment and is not the central causal claim of the paper. Our results suggest that temporal smoothing is the primary requirement, while block/neuron scaling can improve the frontier in some settings.

4.4. Full-state diagnostic controls

We use full-state variants only for diagnosis. MESH-FULL stores the same unnormalized expert momentum $H_{e,t}$ explicitly in optimizer state rather than in the gradient buffer. MESH-B-FULL additionally stores the block statistic used by MESH-B. Comparing hidden and full-state variants isolates the systems effect of hidden momentum from the optimizer effect of temporal smoothing. In our experiments, full-state variants are often stronger quality diagnostics, while MESH and MESH-B are the memory-efficient algorithms.

5. EXPERIMENTS

5.1. Setup

We use the HuggingFace nanowhale training codebase, which implements a 110M-parameter DeepSeek-V4-style MoE language model [11]. The model has eight layers, four routed experts and one shared expert per layer, hidden size 320, and an untied 129,280-token vocabulary. The vocabulary matrices

Stateless expert Sinkhorn	MESH: average before normalization
$G_{e,t} \rightarrow \text{Sinkhorn}(G_{e,t}) \rightarrow \text{update}$	$H_{e,t} = \beta_1 H_{e,t-1} + G_{e,t} \rightarrow \text{Sinkhorn}(H_{e,t}) \rightarrow \text{update}$
Uses the instantaneous routed-expert gradient. Routing fluctuations are normalized into updates.	Uses a temporally smoothed expert signal. Routing fluctuations are averaged before matrix normalization.
MESH-B additionally multiplies the Sinkhorn direction by a coarse mean-normalized block/neuron inverse-RMS factor.	

Fig. 1. Core MESH mechanism. The failure is an ordering issue: normalizing each instantaneous routed-expert gradient is not equivalent to normalizing a temporally averaged expert signal.

MESH family role assignment.		
Parameter class	Optimizer role	Persistent state
Embedding / LM head	SAGE	light state
Non-expert dense matrices	Sinkhorn	none
Routed/shared experts	MESH	hidden buffer
Optional expert block scale	MESH-B block RMS	small block state
Norms / biases	SAGE	light state

Fig. 2. MESH keeps the SAGE/Sinkhorn hybrid for non-expert parameters and replaces only the MoE expert update. MESH is hidden-momentum expert Sinkhorn; MESH-B adds optional block preconditioning. Full-state variants are diagnostic controls.

are unusually large for this model size: input embedding plus LM head contain 82.75M of 110.37M trainable parameters.

Unless noted, runs train from scratch on FineWeb-Edu for 5,000 optimizer steps with sequence length 2048, per-device batch size 8, gradient accumulation 4, and eight data-parallel workers. This corresponds to an estimated 524,288 tokens per optimizer step and 2.62B model tokens per run. Evaluation uses 512 held-out examples. Because several differences are in the 0.01–0.02 loss range, we interpret small gaps as suggestive rather than conclusive.

SAGE/Sinkhorn and MESH configurations follow the official SAGE-style hyperparameters unless stated otherwise: learning rate 2×10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.99$, weight decay 0.01, cosine schedule, 10% warmup, Sinkhorn scale $\tau = 10$, and $K = 5$ normalization rounds. The AdamW baseline uses the nanowhale-tuned configuration used in the project logs: learning rate 6×10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, cosine schedule, and 3% warmup. We report evaluation loss, mean token accuracy, persistent optimizer-state memory, and peak PyTorch CUDA allocation.

5.2. Main results

Table 1 shows that MESH-B achieves a strong memory-quality trade-off but does not outperform AdamW in the additional seeds. Relative to AdamW, MESH-B reduces optimizer-state memory by 62.5% and peak PyTorch CUDA allocation by about 12.6%, but trails AdamW by about 0.049 evaluation loss. MESH without the block multiplier is close

to MESH-B, suggesting that hidden temporal smoothing is the dominant ingredient and that block preconditioning is an incremental improvement in these seeds.

Table 2 separates diagnostic seed-42 ablations from the multi-seed table. In this seed, full-state MESH-B variants recover AdamW-like quality with about half of AdamW’s optimizer-state memory. We use these runs as mechanistic evidence rather than as final multi-seed performance claims.

5.3. Where the SAGE/Sinkhorn hybrid fails

Table 3 shows that routed experts are the dominant failure point. Converting only routed experts to AdamW recovers most of the gap, while converting only the shared expert helps less. Applying SAGE to all experts worsens training, indicating that the embedding-oriented SAGE update is not a suitable expert replacement.

5.4. What the expert update needs

Table 4 shows that adding a temporal first moment to expert Sinkhorn is the major improvement. V-only, sign-only, and factored-v controls are poor. Block/neuron preconditioning can improve the best single-seed results, but the closeness of MESH and MESH-B in Table 1 prevents a stronger claim that block preconditioning is universally required.

5.5. Memory accounting

AdamW stores two full moment states for all 110.37M parameters, producing 0.883GB optimizer state. Direct SAGE/Sinkhorn stores state mainly for the 82.75M vocabulary/SAGE parameters, producing 0.331GB. MESH keeps this same optimizer-state size: the expert first moment is carried by the gradient-buffer lifecycle and counted separately as a 0.098GB hidden buffer, not as optimizer state. Although the hidden buffer is excluded from optimizer-state accounting because it lives in the gradient-buffer lifecycle, it is still part of the persistent training memory and is reflected in CUDA peak measurements. MESH-B adds only small block statistics at this model size, so its reported optimizer-state memory remains 0.331GB at three-decimal precision. MESH-B-FULL stores the expert first moment explicitly,

Table 1. Main multi-seed comparison. These are the runs for which post-fix hidden-momentum results are available at seeds 100 and 512. MESH is hidden-momentum Sinkhorn without block preconditioning. MESH-B is MESH with a block/neuron inverse-RMS multiplier. State denotes persistent optimizer-state memory.

Optimizer	Seeds	Eval loss	Eval acc.	State (GB)	Peak CUDA (GB)
AdamW	100/512	3.5877 avg	0.3515 avg	0.883	4.375–4.376
MESH-B	100/512	3.6365 avg	0.3436 avg	0.331	3.822–3.823
MESH	100/512	3.6430 avg	0.3430 avg	0.331	3.823–3.824

Table 2. Seed-42 diagnostic comparison. These single-seed ablations should not be directly averaged with Table 1; they identify mechanisms.

Optimizer	Eval loss	State (GB)
AdamW	3.6427	0.883
SAGE/Sinkhorn exact	3.8265	0.331
Experts AdamW	3.6158	0.528
MESH-FULL	3.6619	0.429
MESH-B-FULL	3.6187	0.430
MESH-B-FULL with $V(H)$	3.6102	0.430
Routed-only MESH-B-FULL	3.6353	0.410

Table 3. Expert role ablation, seed 42.

Expert treatment	Eval loss	State (GB)
All experts Sinkhorn	3.8265	0.331
Routed experts AdamW	3.6297	0.488
Shared expert AdamW	3.6895	0.370
Routed+shared AdamW	3.6158	0.528
Routed+shared SAGE	3.8912	0.430

increasing state to about 0.430GB. Peak CUDA allocation decreases less than optimizer-state memory because parameters, activations, communication buffers, gradient buffers, temporary update tensors, and allocator effects remain.

6. ANALYSIS AND LIMITATIONS

6.1. Why routed experts needed momentum

The routed expert failure is best understood as an ordering problem. Stateless Sinkhorn applies $\text{Sinkhorn}(G_t)$ at every step. MESH applies $\text{Sinkhorn}(H_t)$, where H_t averages the expert gradient sequence over time. Because Sinkhorn is nonlinear, $\text{Sinkhorn}(\mathbb{E}G)$ is not equivalent to $\mathbb{E}[\text{Sinkhorn}(G)]$. For routed experts, this difference is large: averaging first filters routing-induced temporal noise, and only then applies matrix normalization. This explains why momentum Sinkhorn improves substantially over instantaneous Sinkhorn, even before adding a block multiplier.

The need for momentum is specific to the routed path. A shared expert is active for every token and behaves more like a dense MLP path. In contrast, a routed expert sees a

Table 4. Expert update decomposition, seed 42. These results motivate MESH but do not imply that every component is universally necessary.

Expert update	Eval loss	State (GB)
Instant Sinkhorn	3.8265	0.331
V-only AdamW	3.8457	0.429
Lion / sign-momentum	4.0046	0.429
Factored-v only	4.2308	0.331
MESH-FULL	3.6619	0.429
MESH-FULL, $\beta_1 = 0.7$	3.6438	0.429
MESH-B-FULL	3.6187	0.430
MESH-B-FULL, $V(H)$	3.6102	0.430

router-conditioned token subset with learned gate weights. Its gradient is therefore a sequence of conditional samples rather than a sequence of dense minibatch estimates. Momentum tracks the expert’s persistent direction across these samples.

6.2. What block preconditioning contributes

Block/neuron inverse-RMS scaling improves some single-seed settings. In seed 42, MESH-B-FULL reaches 3.6187 eval loss, and the $V(H)$ variant reaches 3.6102. However, in seeds 100 and 512, hidden-momentum Sinkhorn without block preconditioning is close to the block-preconditioned MESH-B result. We therefore treat block preconditioning as a useful coarse adaptive scale, not as a conclusively necessary component. The safest conclusion is that MoE expert optimization primarily requires temporal smoothing; block/neuron adaptivity can improve the frontier and deserves further study.

6.3. Residual gap to AdamW

The strongest AdamW seeds remain better than MESH or MESH-B in the current experiment. This residual gap may not be caused by expert optimization alone. The nanowhale configuration is vocabulary-heavy: input embedding plus untied LM head account for approximately 75% of trainable parameters. MESH leaves those matrices on SAGE, while AdamW adapts them with full first and second moments. Non-expert dense matrices also remain on stateless Sinkhorn. Future role ablations should therefore keep MESH on experts

while assigning only vocabulary matrices or only non-expert dense matrices to AdamW.

6.4. Limitations

The study is intentionally narrow. It uses one 110M MoE architecture, one streaming corpus, 5,000-step runs, and primarily 512-example evaluations. Several gaps are small enough that larger evaluation sets and more seeds are needed. We also do not claim that MESH outperforms AdamW. Rather, MESH provides a substantially lower-state point on the memory-quality frontier and identifies temporal smoothing before matrix normalization as a key missing ingredient for MoE expert Sinkhorn optimization.

7. CONCLUSION

We presented MESH, a hidden-momentum Sinkhorn optimizer for MoE expert matrices, and MESH-B, an optional block-preconditioned variant. The study shows that direct SAGE/Sinkhorn optimization fails on routed MoE experts because the expert gradient sequence is conditional and temporally noisy. The missing ingredient is primarily temporal smoothing before matrix normalization. Hidden momentum supplies this signal without storing an explicit expert first-moment tensor in optimizer state. The resulting algorithms reduce optimizer-state memory substantially relative to AdamW while recovering much of the quality lost by stateless Sinkhorn. The remaining gap to AdamW likely involves non-expert role assignment, especially vocabulary matrices, and motivates future work on memory-efficient updates for the entire MoE parameter taxonomy.

8. REFERENCES

- [1] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations*, 2015.
- [2] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [3] R. Sinkhorn, “A relationship between arbitrary positive matrices and doubly stochastic matrices,” *Annals of Mathematical Statistics*, vol. 35, no. 2, pp. 876–879, 1964.
- [4] M. Scetbon, C. Ma, W. Gong, and E. Meeds, “Gradient multi-normalization for efficient LLM training,” *OpenReview*, 2025. Available: <https://openreview.net/forum?id=oanhUGY6un>
- [5] W. Lee and H.-T. Kim, “SAGE: Sign-adaptive gradient for memory-efficient LLM optimization,” *arXiv:2604.07663*, 2026.
- [6] M. A. Sadeghi, “Eliminating the first moment state in Adam optimizer,” *OpenReview*, 2025. Available: <https://openreview.net/forum?id=xRxh480AAM>
- [7] Y. Zhang *et al.*, “Adam-mini: Use fewer learning rates to gain more,” *arXiv:2406.16793*, 2024.
- [8] N. Shazeer *et al.*, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations*, 2017.
- [9] D. Lepikhin *et al.*, “GShard: Scaling giant models with conditional computation and automatic sharding,” in *International Conference on Learning Representations*, 2021.
- [10] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, 2022.
- [11] Hugging Face, “nanowhale: A 110M-parameter DeepSeek-style language model,” GitHub repository, 2025. Available: <https://github.com/huggingface/nanowhale>

A. ADDITIONAL EXPERIMENTAL RESULTS

A.1. Full result tables

Table 5 contains the multi-seed comparison used in the main text. Table 6 collects single-seed expert-update ablations. Table 7 collects routing controls. Table 8 lists negative or partial causal interventions. AdamW baselines use learning rate 6×10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, cosine decay, and 3% warmup. SAGE/MESH-style runs use learning rate 2×10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.99$, weight decay 0.01, cosine decay, 10% warmup, Sinkhorn scale 10, and five Sinkhorn rounds unless otherwise stated.

Optimizer	Seed	Eval loss	Eval acc.	State GB	CUDA peak GB
AdamW	42	3.6427	0.3464	0.883	4.426
AdamW	100	3.5848	0.3520	0.883	4.376
AdamW	512	3.5906	0.3510	0.883	4.375
MESH-B	100	3.6335	0.3450	0.331	3.822
MESH-B	512	3.6395	0.3422	0.331	3.823
MESH	100	3.6364	0.3433	0.331	3.824
MESH	512	3.6495	0.3426	0.331	3.823

Table 5. Post-fix hidden-momentum multi-seed comparison. MESH denotes hidden-momentum expert Sinkhorn without block preconditioning; MESH-B denotes the optional block-preconditioned variant.

Expert update	Eval loss	State GB	Interpretation
SAGE/Sinkhorn exact	3.8265	0.331	Stateless expert Sinkhorn fails.
Routed expert AdamW	3.6297	0.488	Routed experts explain most of the expert gap.
Shared expert AdamW	3.6895	0.370	Shared expert matters less.
Routed+shared AdamW	3.6158	0.528	Strong expert-only upper bound.
Experts SAGE	3.8912	0.430	Embedding-oriented SAGE is unsuitable for experts.
V-only AdamW	3.8457	0.429	Second moment without first moment is insufficient.
Lion / sign-momentum	4.0046	0.429	Sign-only update is insufficient.
Factored-v only	4.2308	0.331	Row/column factored scale alone is too weak.
Block AdamW neuron	3.6268	0.430	Block/neuron scale with momentum is strong.
MESH-FULL	3.6619	0.429	Temporal smoothing explains much of the recovery.
MESH-FULL $\beta_1 = 0.7$	3.6438	0.429	Smoothing strength matters.
MESH-B-FULL $p = 0.5$	3.6187	0.430	Strong full-state diagnostic.
MESH-B-FULL $V(H)$	3.6102	0.430	Best seed-42 diagnostic variant.

Table 6. Expert optimizer decomposition, seed 42 unless marked otherwise. Full-state variants are diagnostic controls; the proposed memory-efficient algorithms are hidden-momentum variants in Table 5.

Routing mode	Optimizer	Eval loss	State GB	Peak GB
Top-1 learned	AdamW	3.6777	0.883	4.198
Top-1 learned	SAGE/Sinkhorn	3.8710	0.331	3.644
Top-2 learned	AdamW	3.6427	0.883	4.426
Top-2 learned	SAGE/Sinkhorn	3.8265	0.331	3.877
Top-3 learned	AdamW	3.5816	0.883	4.553
Top-3 learned	SAGE/Sinkhorn	3.7590	0.331	4.000
All-top-k learned	AdamW	3.6281	0.883	4.777
All-top-k learned	SAGE/Sinkhorn	3.8004	0.331	4.225
All-uniform	AdamW	3.6337	0.883	4.639
All-uniform	SAGE/Sinkhorn	3.7563	0.331	4.036
Random balanced top-2	SAGE/Sinkhorn	3.8050	0.331	–

Table 7. Routing and dense-like controls. Top-k and routing interventions do not remove the SAGE/Sinkhorn gap.

Intervention	Eval loss	State GB	Takeaway
Expert Sinkhorn scale 5	3.8212	0.331	Not just an update-scale issue.
Scale-preserved Sinkhorn	3.7632	0.331	Re-inserting raw gradient scale is insufficient.
Confidence damping	3.7605	0.331	Route confidence is secondary.
Confidence amplification	diverged	0.331	Direct route-confidence amplification is unsafe.
Fused routed Sinkhorn	3.7662	0.331	Expert tensor partitioning is not primary.
Fused routed+shared, all-uniform	3.7180	0.331	Fusion helps slightly but is not enough.

Table 8. Negative and partial interventions.

B. PSEUDOCODE AND IMPLEMENTATION DETAILS

B.1. Sinkhorn normalization

Algorithm 1: SinkhornNormalize(G, K, ϵ)

```

1:  $X \leftarrow G$ 
2: for  $i = 1$  to  $K$  do
3:    $X \leftarrow X / (\|X_{r:}\|_2 + \epsilon)$  {row normalization}
4:    $X \leftarrow X / (\|X_{:c}\|_2 + \epsilon)$  {column normalization}
5: end for
6: return  $X$ 

```

B.2. MESH expert step

Algorithm 2: MESH expert update

Require: expert parameter W , hidden stash H , learning rate η , weight decay λ , Sinkhorn scale γ

```

1: Before backward: if  $H$  exists, set  $\text{grad}(W) \leftarrow \beta_1 H$ ; otherwise set  $\text{grad}(W) \leftarrow 0$ .
2: Run backward. The gradient slot now contains  $H_t = \beta_1 H_{t-1} + G_t$ .
3:  $D_t \leftarrow \text{SinkhornNormalize}(H_t, K, \epsilon)$ .
4:  $W \leftarrow W - \eta\gamma D_t - \eta\lambda W$ .
5: Stash  $H \leftarrow \text{detach}(H_t)$  outside the optimizer-state dictionary.
6: Clear the gradient slot normally.

```

B.3. Optional MESH-B block-preconditioned expert step

Algorithm 3: MESH-B expert update

Require: expert parameter W , hidden accumulator H_t , block state V , power p

```

1:  $D_t \leftarrow \text{SinkhornNormalize}(H_t, K, \epsilon)$ .
2:  $B_t \leftarrow \rho(H_t)$ , where  $\rho$  is row RMS for  $W_1/W_3$  and column RMS for  $W_2$ .
3:  $V \leftarrow \beta_2 V + (1 - \beta_2) B_t^2$ .
4:  $P \leftarrow \text{clip} \left( \frac{(V+\epsilon)^{-p/2}}{\text{mean}((V+\epsilon)^{-p/2})}, c_{\min}, c_{\max} \right)$ .
5:  $W \leftarrow W - \eta\gamma(P \odot D_t) - \eta\lambda W$ .

```

We use H_t rather than a separate raw-gradient stream for the default block statistic. This matches the strongest seed-42 diagnostic, denoted $V(H)$ in the tables, and avoids storing an additional full raw-gradient stream in the hidden-momentum implementation.

B.4. Full-state diagnostic variants

The full-state diagnostics, MESH-FULL and MESH-B-FULL, store H_t explicitly in optimizer state rather than through the gradient buffer. MESH-FULL is otherwise identical to Algorithm 2, while MESH-B-FULL is otherwise identical to Algorithm 3. We use these variants to separate the value of temporal smoothing from the engineering question of whether hidden momentum can faithfully reproduce the explicit first moment.

B.5. Hidden momentum scale convention

The hidden buffer obeys $H_t = \beta_1 H_{t-1} + G_t$, while a conventional Adam-style first moment uses $M_t = \beta_1 M_{t-1} + (1 - \beta_1) G_t$. Thus H_t is an unnormalized first-moment accumulator. Since Sinkhorn normalization and mean-normalized block multipliers are invariant to global scaling up to the explicit update scale, we use H_t directly.

B.6. Trainer lifecycle

A naive `optimizer.zero_grad()` override was insufficient in our training stack because the Trainer, Accelerate, or distributed wrapper can clear gradients through model-level paths. The final implementation uses a prepare-before-backward hook. The optimizer stashes hidden expert buffers after each step; immediately before backward, the trainer reattaches the decayed stash to the gradient slot. Diagnostics on full runs report approximately one prepare call per optimizer step, buffer-shadow cosine near one, and relative error around 10^{-8} .

B.7. Role counts in the main MESH configurations

In the main MESH configuration, 85 tensors with 82.747M parameters use SAGE, 65 non-expert dense tensors with 3.046M parameters use plain Sinkhorn, and 120 expert tensors with 24.576M parameters use hidden-momentum expert Sinkhorn. In MESH-B, the same 120 expert tensors additionally use the block/neuron multiplier. The hidden expert buffer is estimated at 0.098GB. MESH-B has the same expert coverage and adds small block statistics. The optimizer-state footprint is 0.331GB at three-decimal precision, dominated by the vocabulary/SAGE states.